



Use AI to improve your Python code

In your work as a data analyst, you can leverage generative AI tools such as [Gemini](#), [ChatGPT](#), and [Microsoft Copilot](#) to improve your Python code and increase your productivity by automating common data analysis tasks. In this reading, you'll discover how you can use AI to enhance the quality of your Python code throughout your workflow, from generating, debugging, and optimizing code, to handling multiple data sources, and serving as a resource for learning.

Use AI to power Python

Generate code

A key benefit of using AI is its power to generate entire Python functions, classes, or code blocks from a simple natural language prompt. When starting a new project or facing a common data manipulation task, AI can quickly provide useful Python code. Rather than spending time searching for the correct libraries and syntax, you can simply prompt an AI tool to generate the desired code.

For example, if you need to clean a dataset, you can ask an AI tool to generate Python code that reads a CSV file into a pandas DataFrame, removes null values, and corrects data types. AI can output a well-structured script that accomplishes this task and also includes helpful comments that explain each step. Using AI can save you valuable time and improve the accuracy and efficiency of your Python code.

Note: Always remember that AI-generated code must be tested and validated to ensure it is accurate, secure, and performs as expected.

Handle multiple data sources

One of Python's strengths is its ability to handle data from various sources. AI can help streamline this process by generating code to read and aggregate that data. For instance, you might need to combine information from CSV files, a SQL database, and a web API. By writing an effective prompt, you can get a Python script that reads data from all these sources and merges them into a single, clean DataFrame.

For example, consider the following prompt:

I'm a junior data analyst with basic knowledge of Python and the pandas library. I need help creating the base outline for a Python script that automates loading data from various sources (specifically, PDFs and Google Sheets). I want the code to scan each document, organize the data into a DataFrame, and produce one single, organized Google Sheet that does not include duplicate data.

In response, an AI tool can generate a basic outline for the script, complete with explanations for each step. AI might also ask clarifying questions about your data's structure to provide a more tailored response, and offer suggestions for handling potential issues, such as inconsistencies between the data sources. This feedback will help you build a more robust solution.

Debug code

Debugging is often a time-consuming part of coding. AI can dramatically accelerate this process by quickly identifying and suggesting fixes for common Python errors, such as syntax mistakes or logical errors in data manipulation. If your Python script throws an error, simply provide the AI tool with the error message and the relevant code and ask for an explanation. AI can then explain the root cause of the error in plain language and suggest a solution. For instance, if you get an error while trying to join two DataFrames,

AI might point out a misspelled column name or the use of an incorrect join type. By leveraging AI, you can avoid spending time deciphering cryptic error messages and quickly learn how to resolve coding issues.

Optimize code

AI can help you produce cleaner, more robust, and more reliable Python code through various code optimization techniques. For example:

- **Refactor code:** An AI tool can take a functional but perhaps messy or overly complicated Python script and suggest refactored versions that adhere to Python best practices, such as using list comprehensions or simplifying complex conditional logic. This makes your code more readable and maintainable.
- **Improve style:** AI tools can improve your style through automatic code formatting. AI parses your code and rewrites it to conform to a specific style guide, most commonly [PEP 8](#), the official style guide for Python.
- **Enhance performance:** For computationally intensive tasks, AI can analyze your Python code and suggest more efficient data structures or algorithms to improve execution speed.
- **Generate documentation:** An AI tool can automatically generate clear and professional docstrings for your functions. This saves time and ensures your code is well-documented, making it easier for others (or your future self) to understand.
- **Generate unit tests:** An AI tool can automatically generate unit tests for your Python functions, covering successful cases, edge cases, and error handling scenarios. This is a crucial time-saving step for improving the quality and reliability of your Python code.

Learn more about coding

An AI tool can act as an always-available tutor and expert consultant, accelerating your learning and research. When you encounter complex Python code, you can ask AI to

explain it line by line or summarize its purpose in plain language. For questions about Python concepts, standard library functions, or external package usage (like how to use a specific pandas method), AI can provide detailed answers and code examples. You can also use AI as a coding reference to ask how to implement a specific feature, suggest the best library for a task (like web scraping), or review best practices for writing Pythonic code. AI can be an invaluable aid for learning new coding concepts and developing new Python skills for data analysis.

Continue to learn about Python and refine your skills. The more you understand about programming, the more effectively you'll be able to leverage AI as a powerful partner.

Key takeaways

Integrating generative AI into your workflow will significantly enhance your productivity and elevate the quality of your Python coding. By leveraging AI, you can:

- Accelerate data cleaning and preparation tasks.
- Tackle more complex, multi-source data projects with greater ease.
- Reduce time spent troubleshooting errors, freeing you up for deeper analysis.

You don't need to be a Python expert to begin harnessing the power of AI. The key is to start experimenting. Like any new technology, the best way to learn is to try, test, and play. Think about a repetitive task that currently consumes a lot of your time and try using an AI tool to help. As you continue to experiment, you'll learn how to best utilize this powerful resource in your role as a data analyst.

Additional resources

AI tools are often integrated directly into popular Integrated Development Environments (IDEs) such as VS Code and PyCharm. This allows you to interact with AI without leaving your coding environment. Explore tools such as [Gemini Code Assist](#) and [GitHub Copilot](#) to learn more.